

Module 06

Playwright — Browser Automation

Control real browsers programmatically — scraping, testing, and more

ContentAutomation2 — Backend Learning Series

Zero Prerequisites → Production-Ready Code

1. What is Browser Automation?

Browser automation means controlling a web browser (Chrome, Firefox, Edge) programmatically. Instead of you clicking and typing, Python does it. Use cases: web scraping (when sites block simple HTTP requests), automated testing, taking screenshots, filling forms, logging into sites.

Playwright is Microsoft's modern browser automation library. It supports Chromium, Firefox, and WebKit. It is faster and more reliable than Selenium. Crucially, it has a first-class async Python API.

2. Install Playwright

```
pip install playwright
playwright install chromium # download browser binaries (~150 MB)

# For MS Edge specifically:
playwright install msedge
```

3. Your First Script — Open a Page and Extract Text

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    # Launch Chrome headlessly (no visible window)
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    # Navigate to a URL
    page.goto("https://example.com")

    # Extract the page title
    title = page.title()
    print(f"Title: {title}")

    # Extract text from a CSS selector
    heading = page.inner_text("h1")
    print(f"Heading: {heading}")

    # Extract all links
    links = page.eval_on_selector_all(
        "a[href]",
        "elements => elements.map(e => ({href: e.href, text: e.textContent.trim()}))"
    )
    for link in links[:5]:
        print(link)

    browser.close()
```

4. Async Playwright — For Use with asyncio

```
from playwright.async_api import async_playwright
import asyncio
```

```

async def scrape(url: str) -> str:
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        page = await browser.new_page()
        await page.goto(url, timeout=30_000) # 30 second timeout
        content = await page.inner_text("body")
        await browser.close()
        return content

async def main():
    text = await scrape("https://example.com")
    print(text[:200])

asyncio.run(main())

```

5. Sharing One Browser — Multiple Pages

Launching a browser is expensive (~1 second). Launching a page within an existing browser is cheap (~10ms). For concurrent scraping, launch ONE browser and create multiple pages/contexts from it.

```

async def scrape_many(urls: list[str]) -> list[str]:
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)

        async def fetch_one(url: str) -> str:
            # BrowserContext = isolated session (own cookies, storage)
            context = await browser.new_context()
            page = await context.new_page()
            await page.goto(url, timeout=30_000)
            text = await page.inner_text("body")
            await context.close() # releases resources
            return text

        results = await asyncio.gather(*[fetch_one(u) for u in urls])
        await browser.close()
        return results

```

Note: ContentAutomation2 creates one shared browser per research run and uses new contexts per article. This avoids fingerprint throttling (same browser = same cookies) while keeping browser launch overhead to a minimum.

6. Selecting Elements

```

# CSS selectors
page.locator("h1") # first <h1>
page.locator(".article-title") # by class
page.locator("#main-content") # by id
page.locator("a[href*='/article/']") # attribute contains

# Text content
page.get_by_text("Read more")

# Fill input and click button

```

```

await page.fill("input[name='email']", "user@example.com")
await page.click("button[type='submit']")

# Wait for element to appear (useful for SPAs with async loading)
await page.wait_for_selector(".results", timeout=10_000)

# Get all matching elements
articles = await page.query_selector_all(".article-card")
for article in articles:
    title = await article.inner_text(".title")

```

7. Extracting Content — Getting the Article Text

```

async def extract_article(url: str) -> dict:
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        page = await browser.new_page()

        # Block images and stylesheets to speed up loading
        await page.route("**/*.{png,jpg,gif,css}", lambda r: r.abort())

        await page.goto(url, wait_until="domcontentloaded", timeout=30_000)

        # Extract what we need
        title = await page.inner_text("h1")

        # Get all paragraphs in the article body
        paragraphs = await page.eval_on_selector_all(
            "article p",
            "els => els.map(e => e.textContent.trim())"
        )
        body = "\n".join(paragraphs)

        # Get publication date (if available as a meta tag)
        pub_date = await page.get_attribute(
            "meta[property='article:published_time']", "content"
        )

        await browser.close()
        return {"title": title, "body": body, "date": pub_date}

```

8. Persistent Context — Saving Login Sessions

A persistent browser context saves cookies and localStorage to a directory. This lets you log in once (manually or via automation) and reuse that session in future runs without logging in again.

```

from playwright.async_api import async_playwright

async def login_and_save():
    async with async_playwright() as p:
        # The profile_dir stores cookies/localStorage across runs
        context = await p.chromium.launch_persistent_context(
            "./session_data/livemint",
            channel="msedge",

```

```

        headless=False,    # show the browser so you can log in manually
    )
    page = await context.new_page()
    await page.goto("https://www.livemint.com/login")
    # ... user logs in manually ...
    input("Press Enter after you've logged in")
    await context.close() # cookies saved automatically

async def use_saved_session():
    async with async_playwright() as p:
        context = await p.chromium.launch_persistent_context(
            "./session_data/livemint",
            channel="msedge",
            headless=True,    # use saved cookies, no login needed
        )
        page = await context.new_page()
        await page.goto("https://www.livemint.com/premium-article")
        # Already logged in – can access premium content

```

9. Handling Dynamic Content (SPAs)

```

# For React/Vue/Angular sites that load content asynchronously:

# Option 1: wait for a specific element
await page.wait_for_selector(".article-body", timeout=15_000)

# Option 2: wait for network to be idle
await page.goto(url, wait_until="networkidle", timeout=30_000)

# Option 3: wait for a condition
await page.wait_for_function(
    "document.querySelectorAll('.article p').length > 3"
)

```

10. Practice Exercises

- Exercise 1: Install playwright and chromium. Write a script that opens <https://news.ycombinator.com> and prints the first 10 story titles.
- Exercise 2: Extract all href links from any news site.
- Exercise 3: Block images and CSS before loading — measure the speed difference.
- Exercise 4: Open a login page visibly (`headless=False`). Log in, save the session. Then re-run with the session (`headless=True`) and verify you're logged in.
- Exercise 5: Write an async scraper that fetches 5 URLs concurrently with `gather`.
- Exercise 6: Add a `Semaphore(2)` to the concurrent scraper.
- Exercise 7: Extract the publication date from an article's tag.